



Detecting Collusive Shill Bidding in Commercial Online Auctions

L. A. Gerritse¹ · C. F. A. van Wesenbeeck²

Accepted: 20 September 2022 / Published online: 15 October 2022
© The Author(s) 2022

Abstract

Online auctions are increasingly used as a smart and efficient way to optimise the consumers' and sellers' utility. A recently active field of research is the detection of fraud in online auctions. One of the most difficult types of fraud to detect is collusive shill bidding, where multiple user accounts jointly drive up the bids in an auction. This paper revises the Collusive Shill Bidding Algorithm (CSBD) proposed by Majadi et al. (2019) to develop an algorithm that is applied to a data set from an online auction platform (TBAuctions). We find that our algorithm converges, that computation time can be significantly reduced by appropriate choice of parameters, and we identify Shill Bidding for this data set, although the accuracy of the algorithm cannot be tested because of lack of ground truth values for the data. The paper further discusses steps needed for application of the algorithm to (very) large data sets, using a multiple core server, which despite substantial reduction in computation time would still require too much time to foresee a rapid implementation in real-time.

Keywords Auctions · Belief propagation · Markov random field · Online auctions · Shill bidding

1 Introduction

Since the 1990s, online consumer auction websites like eBay and Yahoo! Auctions became increasingly popular and auctions have been chosen as the main mechanism for dynamic pricing situations, where both the anonymity of creating an account and

✉ C. F. A. van Wesenbeeck
c.f.a.van.wesenbeeck@vu.nl

L. A. Gerritse
l.a.gerritse@dnb.nl

¹ De Nederlandsche Bank, Amsterdam, The Netherlands

² School of Business and Economics, Vrije Universiteit Amsterdam, De Boelelaan 1105, Room 10A84, 1081 HV Amsterdam, The Netherlands

the possibility to create multiple accounts make it easy to commit fraud. Many different types of fraud occurring in online auctions have been identified, from being dishonest about the actual specifications and quality of the product to shill bidding where the seller tries to drive up the price by placing extra bids (Chiu et al., 2011; Kaur and Garg, 2015; Trevathan, 2018; Tsang et al., 2014). This paper's focus is on collusive shill bidding, which is a cooperative type of fraud where multiple shill bidding accounts cooperate to increase the price, without the intention to win the auction.

As auction fraud directly disadvantages the consumers and distorts the working of the auction mechanism and decreases the consumers' trust in the security of the system, extensive efforts are made to detect and punish fraud. Yahoo! Auctions, for example, publishes a blacklist of users that behave suspiciously (Chiu et al., 2011). Chiu et al. (2011) identifies 11 types of fraud, from misrepresenting the value and specifications of the item, to failing to pay for the product or selling stolen goods, and tackle all these types of fraud - and even fraudulent behaviour that is not yet defined or discovered - by using Social Network Analysis to cluster user accounts into groups and labelling bidders as suspicious if they have abnormal transactions with sellers and other bidders. Similarly, Tsang et al. (2014) design the SPAN (Score Propagation over an Auction Network) algorithm that identifies 4 feature pairs for buyers and 4 for sellers that display out-of-the-ordinary behaviour. All users are represented in a Markov Random Field (MRF) with two states, a fraudulent state and an honest state. Using Belief Propagation (BP), the beliefs are updated from the information sent by the neighbouring nodes in order to give a final score to all bidders.

Most types of fraud are quite transparent, meaning that it is clear when you are being scammed. More interesting on the other hand, and the main focus of research into fraud in online auctions, is shill bidding. Shill bidding, according to Trevathan (2018), is 'the act of introducing fake bids into an auction on the seller's behalf in order to artificially inflate an item's price'. As established in the literature review by Majadi et al. (2017), most shill-bidding detection methods find single perpetrators by looking at historical data. Lei (2012) first lists the main characteristics of shill bidders: discrepancy in the average bids of the bidder, the amount of time the bidder is active in the auction, how often the bidder bids and the average response time. Then the paper uses clustering analysis to divide users in an honest and a shill bidding group.

However, one would like to give auctioneers the tools to detect and penalize shill bidders while an auction is still running. This can be achieved by a real-time algorithm that tracks the activities in the auction as it is happening, and can immediately penalise users if fraud is detected. Xu et al. (2009) & Majadi et al. (2017) designed a real-time shill bidding algorithm that achieves this, introducing penalties that the auctioneer can issue depending on the shill bidding score assigned to the bidders at varying stages of the auction. Majadi and Trevathan (2018) extend their research to multiple live auctions. Alternatively, Kaur and Garg (2015) offer a different solution by introducing variable bid-fees for every bid that is made, which can be retrieved if a bidder wins, thus discouraging the shill bidders from bidding. Majadi et al. (2017) extend the model to detect **collusive** shill bidding; multiple user accounts engaging

in joint shill bidding. Majadi, Trevathan and Bergmann (2019) propose a collusive shill bidding algorithm, using a similar approach to Tsang et al. (2014) with machine learning techniques. This algorithm is also used in a proposed real-time procedure (Majadi and Trevathan, 2018) and extended to a situation where there are multiple colluding sellers.

Although these contributions illustrate their working in numerical examples, they remain largely theoretical as they have not yet been tested on proper commercial data sets. As almost every paper in the field points out, it is very difficult to get data from commercial auction platforms. Thus, usually a small number of commercial auctions is analysed in combination with simulated data sets and there is no mention of the computability of the algorithms for large data sets. This paper revises and improves the Collusive Shill Bidding Detection Algorithm (CSBD) proposed by Majadi et al. (2019) to develop an algorithm that is applied to a data set from an existing online auction platform (TBAuctions). In addition, it discusses the steps needed to apply the algorithm to (very) large data sets, using a multiple core server. We find that our algorithm converges, that computation time can be significantly reduced by appropriate choice of parameters, and that extension to (very) large data sets is possible, but would still require substantial computing time, making it unrealistic to foresee a rapid implementation in real-time.

2 Material and Methods

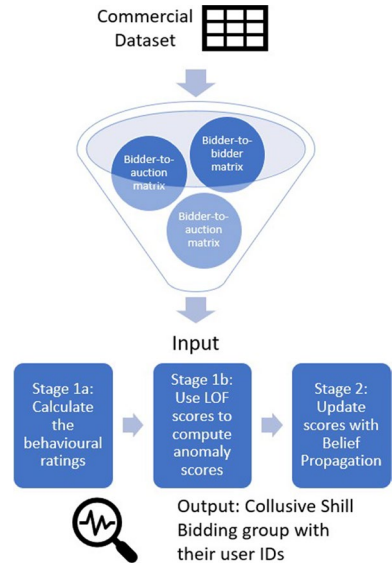
This research uses a data set from the company TBAuctions. This is a Dutch online auction company, with both industrial and consumer auctions. They have provided a data set containing the auctions held in January 2020. TBAuctions modelled the data set to fit into a Spark Sequel data frame and filtered out the outliers and zero values. The data frame consists of a row for every bid in an auction. The columns contain information about the auction ID, the user ID, the time of the bid and whether it was the winning bid. The bids are grouped according to their auction ID, enabling selection of specific auctions from the data frame. The auctions cover the whole spectrum of the markets of TBAuctions and are in different business lines and categories.

In this paper we propose an algorithm to detect collusive shill bidding. For this, machine learning techniques are a necessary tool to map anomalous relations between individual bidders and classify them as shill bidders. We follow the main structure of the Collusive Shill Bidding Detection (CSBD) algorithm from the research by Majadi et al. (2019).

A first understanding and schematic visualisation of the algorithm is given in Fig. 1.

The data set of bids can be viewed as a graph, where the nodes are the bidders, and the edges denote that the bidders have participated in the same auction. The edge-weight is the number of auctions that the bidders have participated in. The total number of bidders and auctions is denoted by n and m respectively. Three scores for each bidder are calculated that can indicate fraudulent behaviour. Next, the Local Outlier Factor (LOF) and Belief Propagation (BP) are applied to analyse the bidding

Fig. 1 The CSBD algorithm



behaviour of every bidder. The output of the algorithm is a group of colluding shill bidders.

3 Theory

In this section we introduce the theoretical framework of the revised CSBD algorithm, which we call the R-CSBD algorithm and is the main contribution of this paper. The main differences between the CSBD and the R-CSBD algorithm are the design of the LOF calculation and the Belief Propagation algorithm. Here, we introduce the concepts theoretically; the implementation is left to the Calculation Sect. (4).

3.1 Indicators

The first feature is the alpha rating(α):

$$\alpha^i = \frac{m^i - w^i}{m}, \quad \text{where } 0 \leq \alpha^i \leq 1. \quad (1)$$

The number of auctions bidder i has made a bid in, is denoted by m^i . The number of auctions bidder i has won is w^i . Lastly, m is the total number of auctions in the dataset. The alpha rating will be close to one for a collusive shill bidder, since they avoid winning any auctions, provided that m is close to m^i . The second rating is the eta rating (η). This rating gives the summation of all the edge-weights of a bidder with its neighbouring bidders. According to Trevathan and Read (2007), collusive

shill bidders have a significantly higher number of edges and edge-weights, resulting in a high eta rating:

$$\eta'_i = \sum_{j=1}^d W_j, \quad (2)$$

which is normalised to yield:

$$\eta_i = \frac{\eta'_i - \eta^{\min}}{\eta^{\max} - \eta^{\min}},$$

where $0 \leq \eta_i \leq 1$.

Here, d is the number of neighbouring nodes of bidder i and W_j is the edge-weight between bidder i and bidder j . Lastly, the lambda rating λ gives an indication of the similarity between the bidding behaviour of two bidders. The beta rating, β_i , is the total number of bids made by bidder i . The lambda rating is then calculated from the beta rating for each possible pair of bidders:

$$\lambda_{i,j} = \begin{cases} 1 & \text{if, } \beta_i = \beta_j \\ \frac{\beta_i}{\beta_j} & \text{if, } \beta_i < \beta_j \\ \frac{\beta_j}{\beta_i} & \text{otherwise,} \end{cases} \quad (3)$$

where $0 \leq \lambda_{i,j} \leq 1$.

The theory assumes that for colluding bidders, the lambda rating will be close to one.

3.2 The Local Outlier Factor

The Local Outlier Factor (LOF) (Breunig et al. (2000), Majadi et al. (2019), Tsang et al. (2014)) is used to calculate the anomaly score. The LOF score is calculated from the distances between a bidder and its k-nearest-neighbours. For each bidder the LOF score is calculated for three behaviour pairs: α & η , α & λ and η & λ . Hence, these pairs are the three different two-dimensional subspaces over which the LOF score is calculated. First, we define the k-nearest neighbourhood for a bidder i and the behaviour pair p :

$$N_{k,p}(i) = \{i' | i' \in D, d(i, i') \leq d_{k,p}(i)\}.$$

In the definition, $d_{k,p}(i, i')$ is the reachability distance between the bidder i and its neighbour i' and $d_{k,p}(i)$ is the distance between bidder i and its k th-nearest neighbour. Then, the Local Reachability Density is computed (LRD), where

$$LRD_{k,p}(i) = \frac{1}{\frac{\sum_{p' \in N_{k,p}(i)} d_{k,p}(i')}{|N_{k,p}(i)|}}. \quad (4)$$

Note that $|N_{k,p}(i)|$ is not necessarily equal to k , as there might be bidders at an equal distance to bidder i as the k th-nearest neighbour. The LOF score for bidder i is then calculated using the LRD scores given by 4 of its neighbours.

$$LOF_{k,p}(i) = \frac{\sum_{i' \in N_{k,p}(i)} \frac{LRD_{k,p}(i')}{LRD_{k,i}(p)}}{|N_{k,p}(i)|} \quad (5)$$

However, whereas the alpha-rating (1) and the eta-rating (2) are individual ratings, the lambda-rating (3) is for a pair of bidders, a fact not acknowledged nor addressed by Majadi et al. (2019). To create a subspace for the α & λ combination and the η & λ combination, a bidder j is fixed which is every bidders' partner in the lambda-rating pair. This is iterated for every possible j . Hence, we have n two-dimensional subspaces for the two lambda behaviour pairs. Finally, the LOF score (5) for each bidder is calculated by first maximizing over j , then over k and lastly over p .

$$\begin{aligned} LOF_{k,p}(i) &= \max_j (LOF_{k,p,j}(i)), \\ LOF_p(i) &= \max_k (LOF_{k,p}(i)), \\ LOF(i) &= \max_p (LOF_p(i)). \end{aligned} \quad (6)$$

3.3 Belief Propagation

The next technique that is used is Belief Propagation within a Markov Random Field (MRF). As explained by Yedidia et al. (2001), the MRF model places all bidders as nodes in an undirected graph, and every bidder is given two nodes. One is the hidden node, which denotes the state that the bidder is in. The other is the observed node, containing a probabilistic value about the state of the bidder.

BP is an important technique in the CSBD algorithm. Majadi et al. (2019) use the term Loopy Belief Propagation, but as pointed out by Murphy et al. (1999), this simply is BP applied on a Loopy Bayesian Network. A major issue is the convergence of BP in loopy systems; e.g. Murphy et al. (1999) report oscillating behavior. Essentially, the lack of convergence is due to the fact that, although BP has a fixed point, additional assumptions on the update algorithm are required to ensure convergence, the most obvious one is to check whether a system wide convex optimization can be defined, with associated gradient (or gradient-related) updating of variable values towards the optimum. Welling and Teh (2013) & Yuille (2002) expand on the fact that the fixed points of BP are the same as those of the Bethe free energy (Yedidia et al., 2001), which suggests alternative algorithms to BP that lead to the same stationary points (Belief Optimization and Concave Convex procedure, respectively). Other approaches place additional bounds on the BP algorithm to ensure convergence (e.g. Ihler and Willsky (2005) apply bounds on the message errors).

The belief propagation constitutes a messaging network that will continue until all messages remain unchanged, and the beliefs for every node are fixed. The messages depend on two functions. The first function is the local evidence function (Yedidia et al., 2001), that gives the statistical dependence between the hidden and observed

node. Following Majadi et al. (2019), we label this function as the prior belief function $\phi_i(x_i)$. The second function is the compatibility function $\psi_{ij}(x_i, x_j)$ that gives the statistical dependence between a node and its neighbours (Majadi et al., 2019). A message from x_i to x_j is given by a summation of the possible states that x_i can be in and a multiplication of the messages that are sent to x_i . In the CSBD model, it is a sum over the bidder in a shill state and the bidder in an honest state:

$$m_{i \rightarrow j}(x_j) = \sum_{x_i} \phi_i(x_i) \psi_{ij}(x_i, x_j) \prod_{k \in N(i) \setminus j} m_{k \rightarrow i}(x_i). \quad (7)$$

After the correspondence, the final belief is calculated for each bidder:

$$b_i(x_i) = \rho \phi_i(x_i) \prod_{j \in N(i)} m_{j \rightarrow i}(x_i). \quad (8)$$

4 Calculation

In the previous section the main ingredients for the algorithm were described. The R-CSBD algorithm has been written using the programming language Python, because it is convenient when working with machine learning and large amounts of data. In this section, a detailed explanation is given of all the steps in the writing of the R-CSBD.

4.1 The Construction of the R-CSBD Algorithm

Here, we briefly explain the different stages in the algorithm; transforming the data, calculating the behavioural ratings, calculating the Local Outlier Factors, and the verification of the data.

4.1.1 Stage 1: Transform the Data

First, the data was transformed into three matrices: a bidder-to-bidder matrix - where the entries are equal to the edge-weights in the graph - and, two bidder-to-auction matrices. One summarises the number of bids each bidder places in all auctions, the second is filled with either a 1 or a 0, depending on whether the bidder has won or lost the auction.

4.1.2 Stage 2a: Calculate the Behavioural Ratings

The behavioural ratings α , η and λ are calculated as described in Sect. 3.1.

Algorithm 1: Stage 2b: Use LOF to calculate the anomaly scores**Result:** LOF anomaly scores $list_of_anomaly_scores = []$;

Calculate the distances;

The α and η behaviour pair;**for** every edge (u, v) **do**

| calculate the distance;

endFor the pairs $\alpha \& \lambda$ and $\eta \& \lambda$;**for** j in $range(n)$ **do**| **for** every edge (u, v) **do**

| | calculate the distances;

| **end****end**

Now use the LOF technique;

for k in $range(n - 2)$ **do**| $k = k + 3$;| **for** i in $range(n)$ **do**| | calculate $LRD_k(\alpha \& \eta)$ | **end**| **for** i in $range(n)$ **do**| | Find the k - nearest neighbourhood;| | Calculate $LOF_k(\alpha \& \eta)$ | **end**| Now iterate over every neighbour for the pairs $\alpha \& \lambda$ and $\eta \& \lambda$;| **for** j in $range(n)$ **do**| | **for** i in $range(n)$ **do**| | | Calculate $LRD_k(\alpha \& \lambda)$;| | | Calculate $LRD_k(\eta \& \lambda)$;| | **end**| | **for** i in $range(n)$ **do**| | | Find the k -nearest neighbourhood of $\alpha \& \lambda$;| | | Find the k -nearest neighbourhood of $\eta \& \lambda$;| | | Calculate $LOF_{k,j}(\alpha \& \lambda)$;| | | Calculate $LOF_{k,j}(\eta \& \lambda)$;| | **end**| **end**| **for** i in $range(n)$ **do**| | Maximize over j ;| | $LOF_{k,\alpha \& \lambda}(i) = \max_j(LOF_{\alpha \& \lambda,k}(i))$;| | $LOF_{k,\eta \& \lambda}(i) = \max_j(LOF_{\eta \& \lambda,k}(i))$;| **end****end****for** i in $range(n)$ **do**| Then for all three behaviour pairs, maximise over k ;| $LOF_p(i) = \max_k(LOF_{p,k}(i))$;| Append to $list_of_anomaly_scores$ **end****for** i in $range(n)$ **do**

| Lastly, maximise over the three behaviour pairs for every bidder.

| $LOF(i) = \max_p(LOF_p(i))$ **end**

4.1.3 Stage 2b: Use LOF to Calculate the Anomaly Score

Stage 2b calculates the distances between all bidders, in every subspace. For the α & η pair, we have a matrix filled with the distances. For the other two behaviour pairs, such a matrix is made for every neighbour. This means that we iterate over all bidders and make a computation for every possible j in the λ_{ij} rating. Thus, we have n distance matrices for both α & λ and η & λ . Next, the calculation of the LOF scores (5) begins by iterating over all k 's. According to Breunig et al. (2000), "The higher the value of k , the more similar the reachability distances for objects within the same neighbourhood." (page 95) Majadi et al. (2019) have chosen to calculate the LOF score for all k 's ranging from 3 to the number of bidder (n). This paper experiments with decreasing k , to see how much this affects the outcome. Since it is the outer loop, this refinement could significantly improve the speed of the algorithm. For various values of k , the k -nearest neighbourhood is found for each bidder, each behaviour pair and each neighbour (for the λ behaviour pairs). Then the LRD and the LOF are computed (4 and 5). First, the LOF score is maximized over j , then over k and lastly over the three possible behaviour pairs (6). This gives an anomaly score for every bidder (see pseudocode, algorithm 2)

4.1.4 Stage 3: Verification

The input in this stage consists of the anomaly scores that were calculated for each bidder in Stage 2 and the input functions that are used in the Belief Propagation. In the R-CSBD algorithm the prior function $\phi_i(x_i)$ is defined as a two dimensional vector that contains the anomaly score for the bidder. The function gives the local evidence of the observed node of the bidder. First, the anomaly score from stage 2 is normalised. Following Majadi et al. (2019), the shill belief for bidder i is given by o_i^s and the honest belief is given by o_i^h . The score that was the output from stage 2 is equal to o_i' . Then we have:

$$o_i^s = \frac{o_i' - o^{min}}{o^{max} - o^{min}},$$

where $0 \leq o_i^s \leq 1$, and

$$o_i^h = 1 - o_i^s.$$

These beliefs are the input for the prior belief function. The first entry corresponds to the shill state and the second entry to the honest state:

$$\phi_i(x_i) = \begin{bmatrix} o_i^s \\ o_i^h \end{bmatrix}.$$

The compatibility function ψ_i is defined by Majadi et al. (2019) as a two-by-two matrix containing the compatibility of two bidders given by a weight between two neighbouring states (Fig. 2).

Table 1
Compatibility matrix (ψ).

Neighbour state	Node belief	
	<i>Shill</i>	<i>Honest</i>
<i>Shill</i>	$1 - \epsilon_0$	ϵ_0
<i>Honest</i>	ϵ_0	$1 - \epsilon_0$

Fig. 2 Compatability matrix, from Majadi et al., 2019

The R-CSBD follows the research by Majadi et al. (2019) and sets ϵ_0 to be 0.2. The next step is initialising all messages for the belief propagation (given by Eq. 7). The messages are two-dimensional vectors. Its entries correspond to the two states of every bidder. In R-CSBD, the messages are initialised by defining all messages for every edge (u, v) in the graph to be:

$$m_{u \rightarrow v} = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}.$$

This is the first message that is sent by bidder u to bidder v . Subsequently, the algorithm makes a list of all the neighbours of every edge by listing all the bidders with which its edge-weight is equal to 1 or higher. The calculation of the messages follows. First we iterate over every edge (u, v) in the graph. Then the following formula is used:

$$m_{u \rightarrow v}(v) = \sum_u w * \psi_{u,v}(u, v) \phi_u(u) \prod_{k \in N(u) \setminus v} m_{k \rightarrow u}(u). \quad (9)$$

The R-CSBD uses the edge-weight between bidder u and bidder v , w , as a weight for the message between u and v . Furthermore, the vector and matrix calculations are defined in the following way. First we take the vector $\phi_u(u)$ and multiply with the first row of the compatibility matrix, that contains the shill state statistical dependency of bidder u on v (1). Subsequently, the same is done for the second row of the matrix (2). We have:

$$\begin{aligned} 1. & \begin{bmatrix} 0.8 & 0.2 \end{bmatrix} * \begin{bmatrix} o_u^s \\ o_u^h \end{bmatrix}, \\ 2. & \begin{bmatrix} 0.2 & 0.8 \end{bmatrix} * \begin{bmatrix} o_u^s \\ o_u^h \end{bmatrix}. \end{aligned}$$

The output value of 1 will form the first entry of a vector, the output value of 2 the second entry. This vector is then multiplied with all messages sent from the neighbours of bidder u to u , which are also all two-dimensional vectors.

$$m_{u \rightarrow v}(v) = w * \begin{pmatrix} 1. \\ 2. \end{pmatrix} \begin{pmatrix} m_{N_1}^s \\ m_{N_2}^h \end{pmatrix}. \quad (10)$$

We define the multiplication of these vectors as taking a product of all the first entries and a product of all the second entries, which results in another two-dimensional vector. After the construction of each message, it is normalised:

$$m_{u \rightarrow v}(u)' = \begin{bmatrix} m^s \\ m^h \end{bmatrix},$$

$$length = \sqrt{(m^s)^2 + (m^h)^2},$$

$$m_{u \rightarrow v}(u) = \frac{1}{length} * m_{u \rightarrow v}(u)'.$$

The algorithm keeps updating these messages until none of them change anymore and the beliefs have converged to its marginal values.

Algorithm 2: Stage 3: Verification

Result: Collusive Shill Bidder group
 Input: the anomaly scores calculated in stage 2;
for i **in** $range(n)$ **do**
 normalise the anomaly score;
 define ϕ_i
end
for i **in** $range(n)$ **do**
 for j **in** $range(n)$ **do**
 compute $m_{i \rightarrow j}$;
 end
end
 Repeat until the messages stop changing;
for i **in** $range(n)$ **do**
 compute the final normalised belief b_i
end
 $collusive_group = []$;
for i **in** $range(n)$ **do**
 If $b_i > 0.75$:
 append ID to list
 else: bidder i is honest
end

Finally, the beliefs are calculated for each bidder. We have:

$$b_u(u) = \rho_1 \phi_u(U) \prod_{v \in N(u)} m_{v \rightarrow u}(u). \quad (11)$$

We multiply all the messages from the neighbours of the bidder u with $\phi_u(u)$ to get a two-dimensional vector again. Similar to above, ρ is the constant that normalises the belief. To find the members of the collusive group of bidders, we set a threshold above which the bidder is a collusive shill bidder. Majadi et al. (2019) sets the threshold to be 0.75. Subsequently, for every bidder we check if the first entry of the belief vector that gives the belief of the shill state, is bigger than 0.75. If that is the case, the bidder is added to the collusive group.

if $b_u^s \geq 0.75$: add to collusive_group = {}.

4.2 Contributions of the R-CSBD Algorithm

To conclude the calculation section, an overview is given of the main contributions of the R-CSBD algorithm. During the writing of the R-CSBD code, it turned out that the pseudocode in the CSBD algorithm by Majadi et al. (2019) skipped several important steps, proposed a structure that contained mathematical errors and was impossible to implement in the algorithm. Below the three errors are explained: the calculation of the LOF score for the λ behaviour pairs, the calculation of the LRD score and finally the calculation of the belief propagation. Subsequent to the error the solution that the R-CSBD algorithm proposed is set out.

In the second stage of the algorithm, the main alteration made by the R-CSBD algorithm was the way the LOF score was calculated for the λ behaviour pairs (given by Eq. 6). In the CSBD algorithm by Majadi et al. (2019) the calculation of the three behaviour pairs seemed to be homogeneous, which was not possible to implement in practice. We had to propose a method to make computations with the λ ratings that are calculated for each bidder pair. This adds significantly to the complexity of the code. Additionally, the formula for calculating the Local Reachability Density (LRD) was adjusted by adding 1 to the sum in the denominator. Because of the small size of the data sets that are tested, many bidders have the same ratings that cause the distance for the first iterations to be zero, leading to undefined outcomes. While the adjustment does change the outcome of the LOF score, the effect is mitigated by the normalisation. Another solution could be to conditionally set the score equal to a certain value when the denominator is zero, but this would hide the fact that a group of bidders with distance equal to zero might be a collusive group, whose LOF score should be selected as input in the verification stage.

In the third stage of the algorithm, the R-CSBD algorithm has an original approach to calculating the messages in the Belief Propagation (Eqs. 10 and 11). The methods described in the pseudocode by Majadi et al. (2019) was not sufficient to implement the calculation in the R-CSBD algorithm. Using both the theoretical discussion of Belief Propagation by Yedidia et al. (2001) and the algorithm proposed by Tsang et al. (2014), who used very similar techniques in their collusive fraud detection algorithm, this research has decided on the approach described above. The previously mentioned papers defined the messages and beliefs to be vectors of the length of the number of states of the nodes. Combining this with the anomaly scores calculated in stage 2 of the CSBD algorithm, the R-CSBD algorithm could define its prior belief function. The vector multiplication was defined by looking at computability and the form of the output that should be achieved by the algorithm.

5 Results and Discussion

5.1 Results

The R-CSBD algorithm was run on 3 small data sets, extracted from the commercial data set (see Sect. 2); data set 1, data set 2 and data set 3. The data sets were limited in size because of the limited computation time and resources that were available. A full application of a multiple core algorithm is considered outside of the scope of this paper. The data sets consist of all the bids in 2, 4 and 6 auctions, with

$$dataset_1 \subset dataset_2 \subset dataset_3.$$

Data set 1 contains 2 auctions and 148 bidders.

The algorithm had a computing time (wall time) of approximately 2 h and 20 min on a laptop with an Intel i7 processor. One can see the α , η and β ratings of the bidders in Table 1 and the normalised anomaly scores after stage 2 and the shill beliefs in Table 2. From the α rating, we can conclude that there was no winner in these auctions, perhaps because the auction was cancelled before the auction was supposed to close. For the final belief scores, 6 bidders had a shill belief of 1.0 and all the other bidders a belief of 0.0. Thus, these 6 bidders were included in the collusive shill bidding group. For the second experiment, the algorithm was run over data set 2, containing 4 auctions and 183 bidders. The algorithm had a computing time of 9 h for this data set. The α , η and β ratings are shown in Table 3. Additionally, the normalised anomaly scores after stage 2 and the shill beliefs are shown in Table 4. After stage 3, the shill beliefs of 90 bidders converged to 1, while the remaining 91 bidders had a belief of 0, leading to an inclusion of these 90 bidders in the collusive shill bidding group.

Then, data set 3 was evaluated. This data set contained 6 auctions and 314 bidders, which increased the computation time of the algorithm to 5.5 days. The behavioural ratings are shown in Table 5, and the normalised anomaly scores and shill beliefs in Table 6. Here, all shill beliefs converge to 0.0 after stage 3, and thus no collusive shill bidders are assigned in this experiment.

Table 1 Data set 1, Behaviour ratings (selected bidders with the highest β rating, 10 shown from 148 bidders)

Bidder	Alpha	Eta	Beta
118	0,5	1	38
147	0,5	1	19
146	0,5	1	11
35	0,5	1	10
100	0,5	1	10
53	0,5	1	9
38	0,5	0	8
18	0,5	1	7
110	0,5	1	7
15	0,5	1	6

Table 2 Data set 1, LOF and shill belief scores (sorted based on the shill belief, 10 shown from 148 bidders)

Bidder	LOF	Shill belief
8	0,033699	1,0
25	0	1,0
38	0,009221	1,0
39	0,009221	1,0
56	0,009221	1,0
86	0,033699	1,0
1	0,004052	0,0
2	1	0,0
3	0	0,0
4	0	0,0

Table 3 Data set 2, Behaviour ratings (selected bidders with highest β rating, 10 shown from 183 bidders)

Bidder	Alpha	Eta	Beta
148	0,25	0,871795	38,00
181	0,25	0,871795	19,00
4	0,25	0,096154	14,00
42	0,25	0,096154	14,00
74	0,25	0,096154	11,00
180	0,25	0,871795	11,00
47	0,25	0,871795	10,00
130	0,25	0,871795	10,00
158	0,50	1	10,00
174	0,25	0,076923	10,00

Table 4 Data set 2, LOF and shill belief scores (sorted based on the shill belief, 10 shown from 183 bidders)

Bidder	LOF	Shill Belief
2	0,091544	1,0
5	0	1,0
6	0,373597	1,0
7	0,373597	1,0
8	0,373597	1,0
11	0,86231	1,0
12	0,86231	1,0
13	0,464935	1,0
14	0,091544	1,0
18	0,09522	1,0
24	0,473894	1,0

Table 5 Data set 3, Behaviour ratings (selected bidders with highest β ratings, 10 shown from 314 bidders)

Bidder	Alpha	Eta	Beta
248	0,17	0,489209	38,00
135	0,17	0,420863	24,00
302	0,17	0,489209	19,00
5	0,17	0,053957	14,00
66	0,17	0,053957	14,00
123	0,17	0,053957	11,00
265	0,50	1	11,00
301	0,17	0,489209	11,00
78	0,17	0,489209	10,00
224	0,17	0,489209	10,00

Table 6 Experiment 3, LOF and shill belief scores (sorted based on highest LOF scores, 10 shown from 314 bidders)

Bidder	LOF	Shill belief
265	1	0,0
223	0,765295	0,0
229	0,765295	0,0
3	0,678097	0,0
4	0,678097	0,0
6	0,678097	0,0
12	0,678097	0,0
16	0,678097	0,0
18	0,678097	0,0
23	0,678097	0,0
29	0,678097	0,0
36	0,678097	0,0

Lastly, for data set 2 the experiment was run again for a reduced number of k iterations. When dividing the number of k iterations by 2, the results remained unchanged and gave 90 collusive shill bidders. For $\frac{1}{3}$ of the k iterations, the number of collusive bidders reduced to 87. When even further reducing the number of k iterations to $\frac{1}{6}$ of the original iterations, the number of collusive bidders increased to 167.

5.2 Running R-CSBD on a Large Data Set

As became apparent from examining the data set that was provided by TBAuctions and running the experiments in the previous section, some auctions have thousands of bidders that are active in the auction. An increase in the number of bidders in the input of the R-CSBD algorithm has an exponential effect on the time that the algorithm runs, since it increases all the layers of iterations (especially when computing

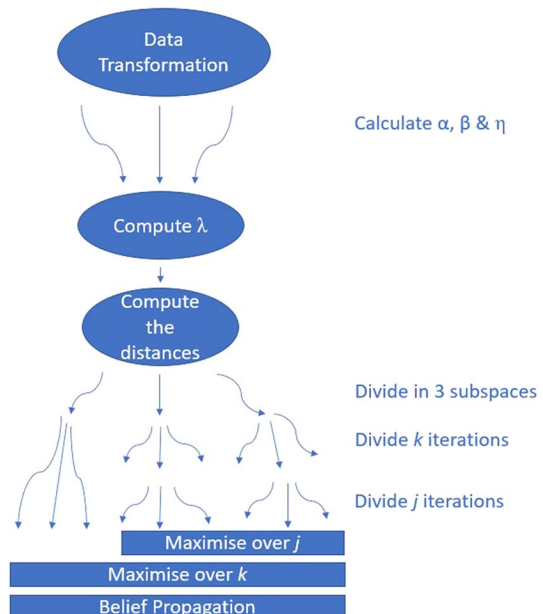
the LOF scores in stage 2). In the results Sect. (5.1), a small data set is run containing 315 bidders. This already takes multiple days to run on a single core, while the data set does not come close to the size of a data set that a company would need to consider in a live set of auctions.

The R-CSBD algorithm, as it is formulated now, is a single thread code; dealing with large amounts of data would require transformation to multi thread algorithms. Hence, the R-CSBD algorithm had to be rewritten to enable processing on multiple cores simultaneously.

As the figure illustrates, we divide the total number of bidders in the data set in subsets, depending on how many cores there are available (Fig. 3). The code will run as a single thread for several parts, and on multiple cores in between. For practical reasons, it is the easiest to perform the transformation of the data set all in a single thread, and make the complete bidder-to-bidder and bidder-to-auction matrices. It is a time efficient procedure and the ratings often depend on its neighbours or summations over an entire auction that would add unnecessary complexity when trying to compute this with multiple cores. Additionally, the λ behaviour rating needs all β ratings to make computations for every pair. Hence, this is also a single threaded part of the code. Another important observation is that for every time that a score is maximized or normalised, all output data needs to be centrally stored such to facilitate joint evaluation. Lastly, the nature of the Belief Propagation technique makes this another single threaded section of the code. The whole network is updating itself by constantly sending messages between neighbouring bidders. Clearly, a single thread approach is needed here.

On the other hand, the many iterations that are computed for every bidder, for every neighbour or for every k value in both the calculation of the behaviour ratings

Fig. 3 R-CSBD in multiple cores



and the LOF scores can easily be split up over multiple cores. Hence, the multiple core extension of the R-CSBD algorithm alternates between single threaded and multiple core parts. The R-CSBD algorithm starts in the same way as before, by transforming the data set into three matrices. Then, the α , η and β ratings are calculated simultaneously for every subset. Subsequently, the ratings are collected (single tread part), and the λ ratings are calculated, Next the LOF scores are computed (multiple treads);the outer iteration over k that denotes the number of nearest neighbours, is separated between subsets and runs simultaneously on multiple cores.

Depending on the size of the data set, it could possibly take several days to compute one k iteration. In that case, it is possible to make more divisions. First we separate the k iteration for all three sub-spaces. We take the first subspace, the behaviour pair α & η , and divide this iteration into subsets. For the other two sub-spaces, we also divide the iteration over the neighbours j into subsets. When all cores are done with their computations, the LOF scores are combined into a single list and form the input for the Belief Propagation stage. This stage is again run in a single thread, but as the BP technique is computationally not demanding, this is not a major bottleneck.

Reducing the number of k iterations would shorten the computation time enormously and additionally reduce the complexity of coding the R-CSBD to run over multiple cores. In the next section, an experiment is done to examine the effect on the output of reducing the number of k iterations. Finally, we estimate the time that is saved by extending the R-CSBD algorithm to multiple cores. The parts that are single threaded are a small fraction of the total computation time. In particular the LOF technique takes up the majority of the computation time. Thus, an estimate for a big data set (on the basis of the smaller experiments that were run), would be that the new improved time comes close to a division of the original R-CSBD computation time by the number of CPU cores that are used to run the multiple core R-CSBD algorithm and even faster on GPU cores.

5.3 Discussion and Recommendations for Further Research

The results, described in Sect. 5.1, do not align with the expectations of this research. Namely, the data sets contain the same auctions, adding 2 every time. While new anomalous behaviour can be brought to light when expanding the data set, it is peculiar that there would be 90 shill bidders in data set 2 but no shill bidders in data set 3. Additionally, it was surprising that the beliefs after stage 3, turn out to be either 0.0 or 1.0, instead of more divergent values. There are several possible explanations for the aforementioned results.

First, the convergence of the belief score of either 0.0 or 1.0 shows that the R-CSBD algorithm works in such a way, that it either classifies bidders as collusive shill bidders or not. There are no values in between. The algorithm probably does not take into account that a bidder could act honestly in certain auctions and dishonest in other auctions. This makes sense, especially if the auctions in the data set are held in diverging categories and business lines where a bidder has no motivation to act fraudulent in all auctions. Increasing the size of the data set and

making it a coherent set of auctions with a correspondence between them, would probably make the beliefs less contrasting. Second, the small size of the data set and the lack of proper selection makes it difficult to find bidders that really stand out, because in the data set the groups of bidders that participate in different auctions barely coincide or can be divergent from each other, resulting in assigning half of the bidders in the data set as collusive shill bidders. Lastly, the behaviour ratings assume that the collusive shill bidders would have a motivation to commit fraud in multiple auctions, which they do not necessarily have in the data sets that were selected for this thesis.

The limitations of the R-CSBD algorithm can be divided into two parts; the accuracy and the efficiency of the R-CSBD algorithm. Because a small data set is used, and we do not know the ground truth values of the bidders, it is not possible to compute an accuracy rate. Additionally, the behaviour ratings are assuming that there is a relation between the sellers of the auctions in the data set, and that the collusive shill bidders have a motivation to drive up the price in multiple auctions. This was discussed with TBAuctions, who indicated that they were interested in finding collusive shill bidders in their Business Lines. A way to find out more about the nature of the bidders after the algorithm selected them for the collusive shill bidding group, is to examine the registration information of the user accounts. An extra indication that might confirm their fraudulent nature, is that the user accounts were made around the same time. This would mean that one person made several accounts to perform collusive shill bidding. Finally, running the algorithm to run a large data set using multiple cores would have given this research the opportunity to share the results with TBAuctions and perform additional analyses, among others adding user account information. This is left for further research.

The second limitation is the efficiency of the R-CSBD algorithm. The part of the second stage where the LOF scores are calculated, takes significantly longer than the other parts of the code. Although this is logical because of the number of iterations that have to be done, there is probably room for improvement. In the CSBD algorithm by Majadi et al. (2019), the number of iterations over k is equal to the number of bidders. In this paper, the effect of lowering k has been examined, as this dramatically decrease the computing time. As discussed in Sect. 5.1, half of the amount of iterations that Majadi et al. (2017) recommend, give the exact same results. Hence, this shows that this is a very promising way to improve the efficiency of the algorithm.

Additional avenues for future research include the following: First, the algorithm still needs to be tested on bigger data sets. Then, as mentioned previously, it would be very interesting and fruitful to extend the algorithm with an extra verification stage. This would entail a close analysis of the bidders that received the highest scores from the R-CSBD algorithm, to see which characteristics they have. Additionally, it could work the other way and make the detection process more efficient. It might also enable auctioneers to easily monitor these basic registration characteristics, and use the R-CSBD algorithm if they see something suspicious.

Secondly, with the additional resources needed to run a large data set, a comparative analysis could be made across the different categories and kinds of auctions to see where collusive shill bidding occurs.

Thirdly, more research might be done into developing real-time detection methods, similar to the research of Majadi et al. (2017). However, for this, the efficiency of the algorithm needs to be improved to allow real-time computation while the auction is in progress, allowing auctioneers to warn users that act suspiciously or exclude them from participating. Lastly, the R-CSBD algorithm might be extended towards detecting other types of fraud or anomalous behaviour, along the lines of Trevathan (2018), where several characteristics of types of in-auction fraud are analysed. If those characteristics are transformed into mathematical ratings, it should be relatively easy to transform the algorithm to detect these other types of fraudulent behaviour as well.

Funding No funding was received for the research reported upon in the paper.

Data Availability Due to the sensitive nature of the data used in this study, TBAuctions does not allow raw data to be shared without their prior consent.

Declarations

Conflict of interest Both authors declare that there is no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). Lof: Identifying density-based local outliers. *SIGMOD Rec*, 29(2), 93–104. <https://doi.org/10.1145/335191.335388>.
- Chiu, C., Ku, Y., Lie, T., & Chen, Y. (2011). Internet auction fraud detection using social network analysis and classification tree approaches. *International Journal of Electronic Commerce / Spring*, 15(3), 123–147. <https://doi.org/10.2753/JEC1086-4415150306>.
- Ihler, A. T., Willsky, A. S., et al. (2005). Loopy belief propagation: Convergence and effects of message errors. *Journal of Machine Learning Research*, 6(May), 905–936.
- Kaur, D., & Garg, D. (2015). Variable bid fee: An online auction shill bidding methodology. In *Proceedings of the IEEE international advance computing conference* (pp. 1–13).
- Lei, B. (2012). A K-means clustering based algorithm for shill bidding recognition in online auction. In *Proceedings of the 24th Chinese control and decision conference* (pp. 939–943).
- Majadi, N., & Trevathan, J. (2018). A real-time detection algorithm for identifying shill bidders in multiple online auctions. In *Proceedings of the 51st Hawaii international conference on system sciences*.
- Majadi, N., Trevathan, J., & Bergmann, N. (2017). A real-time detection of shill bidding in online auction. In *Proceedings of the 15th international conference e-society*.
- Majadi, N., Trevathan, J., & Bergmann, N. (2019). Collusive shill bidding detection in online auctions using Markov random field. *Electronic Commerce Research and Applications*. <https://doi.org/10.1016/j.elerap.2019.100831>.

- Majadi, N., Trevathan, J., Gray, H., Estivill-Castro, V., & Bergmann, N. (2017). Real-time detection of shill bidding in online auctions: A literature review. *Computer Science Review*, 25, 1–18.
- Murphy, K. P., Weiss, Y., & Jordan, M. I. (1999). Loopy belief propagation for approximate inference: An empirical study. In *Proceedings of the 15th conference on uncertainty in artificial intelligence* (pp. 467–475).
- Trevathan, J. (2018). Getting into the mind of an “in-auction” fraud perpetrator. *Computer Science Review*, 27, 1–15. <https://doi.org/10.1016/j.cosrev.2017.10.001>.
- Trevathan, J., & Read, W. (2007). Investigating shill bidding behaviour involving colluding bidders. *Journal of Computers*, 2, 63–75.
- Tsang, S., Koh, Y. S., Dobbie, G., & Alam, S. (2014). SPAN: Finding collaborative frauds in online auctions. *Knowledge-Based Systems*, 71, 389–408. <https://doi.org/10.1016/j.knosys.2014.08.016>.
- Welling, M., & Teh, Y. W. (2013). Belief optimization for binary networks: A stable alternative to loopy belief propagation. In *Proceedings of the 17th conference on uncertainty in artificial intelligence*.
- Xu, H., Bates, C. K., & Shatz, S. M. (2009). Real-time model checking for shill detection in live online auctions. *Software Engineering Research and Practice*, 2009(98), 134–140.
- Yedidia, J. S., Freeman, W. T., & Weiss, Y. (2001). Understanding belief propagation and its generalizations. In *International joint conference on artificial intelligence*.
- Yuille, A. L. (2002). Ccqp algorithms to minimize the Bethe and Kikuchi free energies: Convergent alternatives to belief propagation. *Neural Computation*, 14(7), 1691–1722.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.